

Ex.1 Ecosystem Stability Optimisation with Evolution Strategies

A 2D grid world contains three agent types: carrots (resource), rabbits (herbivores), and eagles (predators). The grid is square with side length L cells. Static obstacles block movement and also block eagle line-of-sight. The simulation is stochastic due to random placement and random tie-breaking in movement. The optimisation goal is to identify initial conditions and an environment configuration that yield a stable, persistent ecosystem over **100 simulation steps**.

World and agents

Map and obstacles

The environment is an $L \times L$ grid where L is an integer in $[\text{MIN_L}, \text{MAX_L}]$.

Obstacles occupy a fraction p_{obs} of the grid, with p_{obs} in $[\text{MIN_POBS}, \text{MAX_POBS}]$.

Obstacles block movement and also block line-of-sight for eagles.

Vision occlusion rule: an eagle can only detect rabbits along unobstructed line-of-sight rays; rabbits behind obstacles are not visible. Occlusion is determined geometrically (ray tracing), avoiding arbitrary exploitability.

Agent states (rabbits and eagles)

Each rabbit and eagle has: age (juvenile/adult), sex (male/female), and energy.

Ageing: juvenile becomes adult after 5 simulation steps.

Vision: eagles can see 5 cells in any direction and rabbits can see only 3 cells in any direction.

Starvation: if energy reaches 0, the agent dies. Energy decreases by 1 each step.

Feeding increases energy:

Rabbits gain energy by eating carrots in their cell.

Eagles gain energy by catching a rabbit in their cell.

Movement

Rabbits move up to 2 cells per step (Chebyshev movement in the provided simulator).

Eagles move up to 3 cells per step.

Movement is blocked by obstacles and boundaries.

Rabbits move toward nearby carrots (if any within local sensing radius), otherwise random walk.

Eagles move toward the nearest visible rabbit; if none visible, random walk.

Tie-breaking uses seeded randomness, preserving stochastic dynamics.

Reproduction

Only adults can reproduce.

Reproduction occurs when an adult male and adult female of the same species are in the same cell.

Reproduction costs energy from both parents (e.g., cost = 3 each).

Offspring spawns into a random empty neighbouring cell in the Moore neighbourhood (radius 1). If no empty cell exists, reproduction fails.

Carrots reproduce asexually with a fixed per-step probability r_c and spawn into a random empty neighbouring cell (radius 1). Carrot reproduction is fixed and not optimised.

Optimisation decision variables (what ES controls)

The ES optimises a real-valued vector θ that is decoded into simulation parameters:

- L : map size, integer in $[MIN_L, MAX_L]$
- p_{obs} : obstacle fraction, real in $[MIN_POBS, MAX_POBS]$
- N_{e0} : initial eagles, integer in $[MIN_EAGLES, MAX_EAGLES]$
- N_{r0} : initial rabbits, integer in $[MIN_RABBITS, MAX_RABBITS]$
- N_{c0} : initial carrots, integer in $[MIN_CARROTS, MAX_CARROTS]$
- ρ_e : initial female ratio for eagles, real in $[MIN_RH0, MAX_RH0]$
- ρ_r : initial female ratio for rabbits, real in $[MIN_RH0, MAX_RH0]$

Objective and constraints

No species may go extinct during the first **100 steps**.

A candidate θ is evaluated across S random seeds (e.g., $S = 5$). For each seed, simulate 100 steps and record:

- T_{ext} : time-to-extinction for each species (if extinction occurs; else 100)
- N_{min} : minimum population of each species over the run
- Oscillation penalty: population volatility (e.g., mean absolute change per step)
- Overcrowding penalty: population density vs free cells

Per-seed fitness:

- If any species goes extinct before 100:
 $fitness_seed = -M \times (100 - \min(T_{ext}))$
- Else:
 $fitness_seed = w1 \times \min_species(N_{min}) - w2 \times volatility - w3 \times overcrowding_penalty$

Robust fitness across seeds:

- $fitness(\theta) = median(fitness_seed \text{ across } S \text{ seeds})$

Student tasks (what you must complete in the notebook)

The simulator is already provided. Your task is to complete the **ES-specific components** marked as `# TODO`.

Step 1 — Run and understand the simulator (provided)

The notebook already contains:

- `GridWorld` (environment, obstacles, movement, feeding, reproduction)
- helper metrics (`extinction_time`, `mean_abs_change`, `overcrowding_penalty`)
- baseline runner and video renderer

Your responsibility is to read the simulator and confirm you understand the update order.

Step 2 — Validate dynamics without optimisation (provided runner)

Use the baseline vector `θθ` and run a few seeds.

Check:

- starvation reduces populations when food/prey is unavailable,
- reproduction only occurs for adults,
- obstacles block movement and eagle line-of-sight.

Document one failure case (e.g., predator die-out) and explain why.

Step 3 — Implement parameter encoding and repair (**TODO: `clamp`, `decode`**)

Implement:

- `clamp(x, lo, hi)`
Ensures $lo \leq x \leq hi$.
- `decode(theta_real)`
Converts the real-valued vector into bounded, valid simulation parameters:
 - clamp values to bounds,
 - round integer parameters (`L`, `N_e0`, `N_r0`, `N_c0`),
 - keep ratios (`ρ_e`, `ρ_r`) in `[MIN_RHO, MAX_RHO]`.

You must verify that decoded parameters always satisfy constraints.

Step 4 — Implement robust fitness evaluation (**TODO: `fitness`**)

Implement `fitness(theta_real, base_seed, S, steps)`:

- decode `theta_real` into `DecodedParams`,
- run `S` seeds using `fitness_one_seed(...)`,
- aggregate with the **median**,
- return a scalar fitness value.

Step 5 — Implement a simple (μ, λ) Evolution Strategy (TODO: `run_es`)

Implement a basic ES:

- keep a mean vector `theta_mean`,
- generate λ offspring using Gaussian mutation,
- evaluate each offspring with `fitness(...)`,
- select the top μ offspring,
- update the mean as the average of the top μ ,
- track and print per-generation:
 - best fitness,
 - median fitness,
 - decoded best parameters.

Step 6 — Run ES and analyse convergence (provided run block)

After completing `run_es`, run the provided pipeline:

- baseline summary,
- ES generation logs,
- best solution summary,
- validation report.

Step 7 — Validate best solution robustly (provided)

Use the provided `validate_solution(...)` to re-evaluate the best θ^* across more seeds and report:

- success rate (no extinction),
- median minimum populations,
- volatility measures.

Step 8 — Render a simulation video of the best parameters (provided)

The notebook already includes:

- `simulate_frames(...)` to capture a 100-step trajectory
- `render_video(...)` to produce an MP4 using the 4×1 layout template

After ES completes, run the rendering code to generate and display the video.

```
#
=====
=====
# Exercise: Ecosystem Stability Optimisation – Notebook (single cell)
#
# Scenario
# A 2D grid world contains three agent types: carrots (resource),
# rabbits (herbivores),
```

```

# and eagles (predators). Obstacles block movement and eagle line-of-
sight. The
# simulation is stochastic due to random placement and tie-breaking.
#
# Goal
# Implement an Evolution Strategy (ES) to search for initial
conditions and environment
# parameters that keep ALL species alive for SIM_STEPS steps and
(ideally) produce
# stable population trajectories.
#
# Student tasks
# - Complete the TODOs in the ES section (decode, fitness aggregation,
ES loop).
# - Run baseline and compare with ES best.
# - Render a short MP4 simulation of the best found parameters.
#
# Tips for the TODO section
# 1) Start by implementing clamp(...) and decode(...); test decode on
theta0.
# 2) Implement fitness(...) using median over S seeds. Print per-seed
scores while debugging.
# 3) In ES, begin with small settings (generations=10, lam=10) to
confirm plumbing,
#     then scale to larger values.
# 4) If no feasible solution appears, inspect failure mode:
#     - Eagles go extinct -> increase rabbit availability or reduce
eagle starvation pressure.
#     - Rabbits go extinct -> reduce eagle density or improve carrot
persistence.
# 5) Keep mutations modest for bounded variables; repair after
mutation is required.
#
# Requirements: numpy, matplotlib, ffmpeg available to matplotlib (for
MP4 writer)
#
=====
=====

from __future__ import annotations

from dataclasses import dataclass
from typing import List, Tuple, Dict, Optional
import numpy as np

import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import matplotlib.lines as mlines
from matplotlib.patches import Rectangle
from matplotlib.animation import FuncAnimation, FFMpegWriter

```

```

from IPython.display import Video, display

#
=====
=====
# CONSTANTS: population bounds + map bounds
#
=====
=====
MIN_EAGLES, MAX_EAGLES = 2, 8
MIN_RABBITS, MAX_RABBITS = 20, 100
MIN_CARROTS, MAX_CARROTS = 400, 600

MIN_L, MAX_L = 40, 60
MIN_POBS, MAX_POBS = 0.05, 0.10
MIN_RHO, MAX_RHO = 0.40, 0.60

SIM_STEPS = 100 # keep short for demo/video; increase for your main
experiments

#
=====
=====
# Utility: Bresenham line for LoS
#
=====
=====
def bresenham_line(x0: int, y0: int, x1: int, y1: int) ->
List[Tuple[int, int]]:
    cells: List[Tuple[int, int]] = []
    dx = abs(x1 - x0)
    dy = abs(y1 - y0)
    sx = 1 if x0 < x1 else -1
    sy = 1 if y0 < y1 else -1
    err = dx - dy

    x, y = x0, y0
    while True:
        cells.append((x, y))
        if x == x1 and y == y1:
            break
        e2 = 2 * err
        if e2 > -dy:
            err -= dy
            x += sx
        if e2 < dx:
            err += dx
            y += sy

```

```

        return cells

#
=====
# Data structures
#
=====
@dataclass
class Agent:
    x: int
    y: int
    species: str # "rabbit" or "eagle"
    sex: int      # 0 male, 1 female
    age_steps: int
    energy: int

    @property
    def adult(self) -> bool:
        return self.age_steps >= 5

@dataclass
class DecodedParams:
    L: int
    p_obs: float
    N_e0: int
    N_r0: int
    N_c0: int
    rho_e: float
    rho_r: float

#
=====
# Grid world simulator
#
=====
class GridWorld:
    def __init__(
        self,
        L: int,
        p_obs: float,
        N_e0: int,
        N_r0: int,
        N_c0: int,

```

```

    rho_e: float,
    rho_r: float,
    seed: int,
    r_c: float = 0.05,
):
    self.L = int(L)
    self.rng = np.random.default_rng(int(seed))
    self.r_c = float(r_c)

    self.obs = np.zeros((self.L, self.L), dtype=bool)
    self._place_obstacles(p_obs)

    self.carrots: set[Tuple[int, int]] = set()
    self.rabbits: List[Agent] = []
    self.eagles: List[Agent] = []

    # Energy model (defined before spawn)
    self.rabbit_energy_init = 8
    self.eagle_energy_init = 10
    self.rabbit_energy_cap = 12
    self.eagle_energy_cap = 16
    self.rabbit_eat_gain = 6
    self.eagle_eat_gain = 8
    self.repro_cost = 3

    # Vision / movement
    self.rabbit_vision = 3
    self.eagle_vision = 5
    self.rabbit_speed = 2          # Chebyshev steps
    self.eagle_speed = 3

    self._spawn_initial(N_c0, N_r0, N_e0, rho_r, rho_e)

    self.pop_history: Dict[str, List[int]] = {"carrot": [],
"rabbit": [], "eagle": []}

    def _place_obstacles(self, p_obs: float) -> None:
        total = self.L * self.L
        n_obs = int(round(float(p_obs) * total))
        idx = self.rng.choice(total, size=n_obs, replace=False)
        xs = idx // self.L
        ys = idx % self.L
        self.obs[xs, ys] = True

    def in_bounds(self, x: int, y: int) -> bool:
        return 0 <= x < self.L and 0 <= y < self.L

    def is_blocked(self, x: int, y: int) -> bool:
        return bool(self.obs[x, y])

```

```

def _occupied_by_any_agent(self, x: int, y: int) -> bool:
    for a in self.rabbits:
        if a.x == x and a.y == y:
            return True
    for a in self.eagles:
        if a.x == x and a.y == y:
            return True
    return False

def is_empty(self, x: int, y: int) -> bool:
    if not self.in_bounds(x, y) or self.is_blocked(x, y):
        return False
    if (x, y) in self.carrots:
        return False
    if self._occupied_by_any_agent(x, y):
        return False
    return True

def _random_empty_cell(self) -> Tuple[int, int]:
    while True:
        x = int(self.rng.integers(0, self.L))
        y = int(self.rng.integers(0, self.L))
        if self.is_empty(x, y):
            return (x, y)

def _spawn_initial(self, N_c0: int, N_r0: int, N_e0: int, rho_r:
float, rho_e: float) -> None:
    # Carrots
    for _ in range(int(N_c0)):
        x, y = self._random_empty_cell()
        self.carrots.add((x, y))

    # Rabbits
    n_rf = int(round(int(N_r0) * float(rho_r)))
    n_rm = int(N_r0) - n_rf
    sexes = [1] * n_rf + [0] * n_rm
    self.rng.shuffle(sexes)
    for s in sexes:
        x, y = self._random_empty_cell()
        self.rabbits.append(Agent(x, y, "rabbit", int(s), 0,
self.rabbit_energy_init))

    # Eagles (optimised)
    n_ef = int(round(int(N_e0) * float(rho_e)))
    n_em = int(N_e0) - n_ef
    sexes = [1] * n_ef + [0] * n_em
    self.rng.shuffle(sexes)
    for s in sexes:
        x, y = self._random_empty_cell()
        self.eagles.append(Agent(x, y, "eagle", int(s), 0,

```

```

self.eagle_energy_init))

# -----
# Movement helpers (Chebyshev with BFS)
# -----
def _reachable_cells(self, start: Tuple[int, int], max_steps: int)
-> List[Tuple[int, int]]:
    sx, sy = start
    q = [(sx, sy, 0)]
    seen = {(sx, sy)}
    out = [(sx, sy)]
    nbrs = [(-1, -1), (-1, 0), (-1, 1),
             (0, -1),          (0, 1),
             (1, -1),  (1, 0),  (1, 1)]
    while q:
        x, y, d = q.pop(0)
        if d == max_steps:
            continue
        for dx, dy in nbrs:
            nx, ny = x + dx, y + dy
            if not self.in_bounds(nx, ny) or self.is_blocked(nx,
ny):
                continue
            if (nx, ny) in seen:
                continue
            seen.add((nx, ny))
            out.append((nx, ny))
            q.append((nx, ny, d + 1))
    return out

def _shortest_path_step(self, start: Tuple[int, int], goal:
Tuple[int, int], max_steps: int) -> Tuple[int, int]:
    if start == goal:
        return start

    sx, sy = start
    gx, gy = goal
    nbrs = [(-1, -1), (-1, 0), (-1, 1),
             (0, -1),          (0, 1),
             (1, -1),  (1, 0),  (1, 1)]

    q = [(sx, sy)]
    parent: Dict[Tuple[int, int], Tuple[int, int]] = {}
    seen = {(sx, sy)}

    while q:
        x, y = q.pop(0)
        if (x, y) == (gx, gy):
            break
        for dx, dy in nbrs:

```

```

        nx, ny = x + dx, y + dy
        if not self.in_bounds(nx, ny) or self.is_blocked(nx,
ny):
            continue
        if (nx, ny) in seen:
            continue
        seen.add((nx, ny))
        parent[(nx, ny)] = (x, y)
        q.append((nx, ny))

    if (gx, gy) not in seen:
        reach = self._reachable_cells(start, max_steps)
        return reach[int(self.rng.integers(0, len(reach)))]

    path = [(gx, gy)]
    cur = (gx, gy)
    while cur != (sx, sy):
        cur = parent[cur]
        path.append(cur)
    path.reverse()
    idx = min(max_steps, len(path) - 1)
    return path[idx]

# -----
# Sensing
# -----
def _nearest_carrot_within(self, x: int, y: int, radius: int) ->
Optional[Tuple[int, int]]:
    best = None
    best_d = 10**9
    for (cx, cy) in self.carrots:
        d = max(abs(cx - x), abs(cy - y))
        if d <= radius:
            if d < best_d:
                best_d = d
                best = (cx, cy)
            elif d == best_d and best is not None:
                if self.rng.random() < 0.5:
                    best = (cx, cy)
    return best

def _eagle_visible_rabbits(self, ex: int, ey: int, radius: int) ->
List[Tuple[int, int]]:
    out: List[Tuple[int, int]] = []
    for r in self.rabbits:
        if max(abs(r.x - ex), abs(r.y - ey)) > radius:
            continue
        line = bresenham_line(ex, ey, r.x, r.y)
        blocked = False
        for (lx, ly) in line[1:-1]:

```

```

        if self.is_blocked(lx, ly):
            blocked = True
            break
        if not blocked:
            out.append((r.x, r.y))
    return out

def _nearest_visible_rabbit(self, ex: int, ey: int, radius: int) -
> Optional[Tuple[int, int]]:
    visibles = self._eagle_visible_rabbits(ex, ey, radius)
    if not visibles:
        return None
    best = None
    best_d = 10**9
    for (rx, ry) in visibles:
        d = max(abs(rx - ex), abs(ry - ey))
        if d < best_d:
            best_d = d
            best = (rx, ry)
        elif d == best_d and best is not None:
            if self.rng.random() < 0.5:
                best = (rx, ry)
    return best

# -----
# Dynamics
# -----
def _record_pops(self) -> None:
    self.pop_history["carrot"].append(len(self.carrots))
    self.pop_history["rabbit"].append(len(self.rabbits))
    self.pop_history["eagle"].append(len(self.eagles))

def _carrot_repro(self) -> None:
    if not self.carrots:
        return
    new_positions: List[Tuple[int, int]] = []
    for (x, y) in list(self.carrots):
        if self.rng.random() < self.r_c:
            candidates = []
            for dx in (-1, 0, 1):
                for dy in (-1, 0, 1):
                    if dx == 0 and dy == 0:
                        continue
                    nx, ny = x + dx, y + dy
                    if self.is_empty(nx, ny):
                        candidates.append((nx, ny))
            if candidates:
                new_positions.append(candidates[int(self.rng.integers(0,
len(candidates)))]))

```

```

        for pos in new_positions:
            self.carrots.add(pos)

    def _move_rabbits(self) -> None:
        for r in self.rabbits:
            target = self._nearest_carrot_within(r.x, r.y,
self.rabbit_vision)
            if target is None:
                reach = self._reachable_cells((r.x, r.y),
self.rabbit_speed)
                if len(reach) > 1:
                    reach = [p for p in reach if p != (r.x, r.y)]
                    r.x, r.y = reach[int(self.rng.integers(0,
len(reach)))]
                else:
                    r.x, r.y = self._shortest_path_step((r.x, r.y),
target, self.rabbit_speed)

    def _move_eagles(self) -> None:
        for e in self.eagles:
            target = self._nearest_visible_rabbit(e.x, e.y,
self.eagle_vision)
            if target is None:
                reach = self._reachable_cells((e.x, e.y),
self.eagle_speed)
                if len(reach) > 1:
                    reach = [p for p in reach if p != (e.x, e.y)]
                    e.x, e.y = reach[int(self.rng.integers(0,
len(reach)))]
                else:
                    e.x, e.y = self._shortest_path_step((e.x, e.y),
target, self.eagle_speed)

    def _interactions(self) -> None:
        # Rabbits eat carrots
        for r in self.rabbits:
            pos = (r.x, r.y)
            if pos in self.carrots:
                self.carrots.remove(pos)
                r.energy = min(self.rabbit_energy_cap, r.energy +
self.rabbit_eat_gain)

        # Eagles catch rabbits
        if not self.rabbits or not self.eagles:
            return

        rabbits_by_cell: Dict[Tuple[int, int], List[int]] = {}
        for i, r in enumerate(self.rabbits):
            rabbits_by_cell.setdefault((r.x, r.y), []).append(i)

```

```

        dead = set()
        for e in self.eagles:
            cell = (e.x, e.y)
            if cell in rabbits_by_cell:
                alive = [i for i in rabbits_by_cell[cell] if i not in
dead]
                    if alive:
                        victim = alive[int(self.rng.integers(0,
len(alive)))]
                        dead.add(victim)
                        e.energy = min(self.eagle_energy_cap, e.energy +
self.eagle_eat_gain)

            if dead:
                self.rabbits = [r for i, r in enumerate(self.rabbits) if i
not in dead]

    def _reproduce_species(self, agents: List[Agent], species: str) ->
None:
        if not agents:
            return

        by_cell: Dict[Tuple[int, int], List[int]] = {}
        for i, a in enumerate(agents):
            by_cell.setdefault((a.x, a.y), []).append(i)

        newborns: List[Agent] = []
        for cell, idxs in by_cell.items():
            males = [i for i in idxs if agents[i].adult and
agents[i].sex == 0 and agents[i].energy >= self.repro_cost]
            females = [i for i in idxs if agents[i].adult and
agents[i].sex == 1 and agents[i].energy >= self.repro_cost]
            if not males or not females:
                continue

            self.rng.shuffle(males)
            self.rng.shuffle(females)
            n_pairs = min(len(males), len(females))

            for k in range(n_pairs):
                mi = males[k]
                fi = females[k]
                agents[mi].energy -= self.repro_cost
                agents[fi].energy -= self.repro_cost

                x, y = cell
                candidates = []
                for dx in (-1, 0, 1):
                    for dy in (-1, 0, 1):
                        if dx == 0 and dy == 0:

```

```

        continue
        nx, ny = x + dx, y + dy
        if not self.in_bounds(nx, ny) or
self.is_blocked(nx, ny):
            continue
            if (nx, ny) in self.carrots:
                continue
            if self._occupied_by_any_agent(nx, ny):
                continue
            candidates.append((nx, ny))
        if not candidates:
            continue

        bx, by = candidates[int(self.rng.integers(0,
len(candidates)))]
        sex = int(self.rng.integers(0, 2))
        init_energy = self.rabbit_energy_init if species ==
"rabbit" else self.eagle_energy_init
        newborns.append Agent(bx, by, species, sex, 0,
init_energy))

        agents.extend(newborns)

    def _reproduction(self) -> None:
        self._reproduce_species(self.rabbits, "rabbit")
        self._reproduce_species(self.eagles, "eagle")

    def _age_and_decay(self) -> None:
        for r in self.rabbits:
            r.age_steps += 1
            r.energy -= 1
        for e in self.eagles:
            e.age_steps += 1
            e.energy -= 1

    def _cull_dead(self) -> None:
        self.rabbits = [r for r in self.rabbits if r.energy > 0]
        self.eagles = [e for e in self.eagles if e.energy > 0]

    def step(self) -> None:
        self._record_pops()
        self._carrot_repro()
        self._move_rabbits()
        self._move_eagles()
        self._interactions()
        self._reproduction()
        self._age_and_decay()
        self._cull_dead()

    def run(self, steps: int = SIM_STEPS, early_stop: bool = True) ->

```

```

Dict[str, List[int]]:
    for _ in range(int(steps)):
        self.step()
        if early_stop:
            if len(self.carrots) == 0 or len(self.rabbits) == 0 or
len(self.eagles) == 0:
                self._record_pops()
                break
        return self.pop_history

#
=====
=====
# Fitness utilities (provided)
#
=====
=====
def extinction_time(series: List[int], max_steps: int = SIM_STEPS) ->
int:
    for t, n in enumerate(series):
        if n <= 0:
            return t
    return max_steps

def mean_abs_change(series: List[int]) -> float:
    if len(series) < 2:
        return 0.0
    diffs = [abs(series[i + 1] - series[i]) for i in range(len(series)
- 1)]
    return float(np.mean(diffs))

def overcrowding_penalty(pop_hist: Dict[str, List[int]], L: int,
p_obs: float) -> float:
    free = int(round(L * L * (1.0 - p_obs)))
    if free <= 0:
        return 1e6
    total = np.array(pop_hist["carrot"]) +
np.array(pop_hist["rabbit"]) + np.array(pop_hist["eagle"])
    ratio = float(np.max(total / free)) if len(total) else 0.0
    return float(max(0.0, ratio - 0.80) * 100.0)

#
=====
=====
# TODO: Evolution Strategy (ES) implementation
#
=====
=====

```

```

# Complete the functions below. After completing, run the notebook
cell.
#
# Expected behaviour:
# - Baseline decoded + baseline final printed
# - ES generation logs printed
# - Best fitness + best decoded printed
# - Validation report printed
# - MP4 saved and displayed
#
=====
=====

def clamp(x: float, lo: float, hi: float) -> float:
    # TODO:
    # Implement clamping so lo <= x <= hi.
    # Tip: use min/max or np.clip.
    raise NotImplementedError

def decode(theta_real: np.ndarray) -> DecodedParams:
    # TODO:
    # Convert a real-valued vector theta_real into bounded parameters.
    # Parameter order: [L, p_obs, N_e0, N_r0, N_c0, rho_e, rho_r]
    # Requirements:
    # - clamp to bounds
    # - round integer parameters: L, N_e0, N_r0, N_c0
    # - keep rho values within [MIN_RHO, MAX_RHO]
    raise NotImplementedError

def fitness_one_seed(params: DecodedParams, seed: int, steps: int =
SIM_STEPS) -> float:
    # Provided. Uses early-stop and a penalty for extinction.
    env = GridWorld(
        L=params.L, p_obs=params.p_obs,
        N_e0=params.N_e0, N_r0=params.N_r0, N_c0=params.N_c0,
        rho_e=params.rho_e, rho_r=params.rho_r,
        seed=seed, r_c=0.05
    )
    pop = env.run(steps=steps, early_stop=True)

    t_c = extinction_time(pop["carrot"], steps)
    t_r = extinction_time(pop["rabbit"], steps)
    t_e = extinction_time(pop["eagle"], steps)
    t_min = min(t_c, t_r, t_e)

    if t_min < steps:
        M = 50.0
        return -float(M) * float(steps - t_min)

    nmin = min(min(pop["carrot"]), min(pop["rabbit"]),

```

```

min(pop["eagle"]))
    vol = mean_abs_change(pop["carrot"]) +
mean_abs_change(pop["rabbit"]) + mean_abs_change(pop["eagle"])
    over = overcrowding_penalty(pop, params.L, params.p_obs)
    return float(1.0 * nmin - 1.0 * vol - 1.0 * over)

def fitness(theta_real: np.ndarray, base_seed: int = 12345, S: int =
5, steps: int = SIM_STEPS) -> float:
    # TODO:
    # Evaluate theta_real robustly across S different seeds.
    # Requirements:
    # - decode theta_real -> params
    # - compute per-seed fitness using fitness_one_seed(...)
    # - aggregate using median across seeds
    raise NotImplementedError

def run_es(
    generations: int = 80,
    mu: int = 10,
    lam: int = 40,
    seed: int = 7,
    S: int = 5,
    steps: int = SIM_STEPS,
    verbose_every: int = 1,
):
    # TODO:
    # Implement a (mu, lambda)-ES.
    # Tips:
    # - Use rng = np.random.default_rng(seed)
    # - Represent theta as a real vector of length 7.
    # - Start with a reasonable mean and per-dimension sigma.
    # - For each generation:
    #   1) sample lam offspring via Gaussian mutation
    #   2) evaluate offspring fitness(...)
    #   3) select top mu, update mean = average(top mu)
    #   4) track and print best and median fitness
    # - Return (best_theta, best_fit, history)
    raise NotImplementedError

#
=====
=====
# Validation and rendering helpers (provided)
#
=====
=====

def validate_solution(theta_real: np.ndarray, seeds: int = 20, steps:
int = SIM_STEPS) -> Dict[str, float]:

```

```

params = decode(theta_real)
scores = []
success = 0
min_c, min_r, min_e = [], [], []
vol_c, vol_r, vol_e = [], [], []

for i in range(int(seeds)):
    base_seed = 500000 + 97 * i
    env = GridWorld(
        L=params.L, p_obs=params.p_obs,
        N_e0=params.N_e0, N_r0=params.N_r0, N_c0=params.N_c0,
        rho_e=params.rho_e, rho_r=params.rho_r,
        seed=base_seed
    )
    pop = env.run(steps=steps, early_stop=True)

    tmin = min(
        extinction_time(pop["carrot"], steps),
        extinction_time(pop["rabbit"], steps),
        extinction_time(pop["eagle"], steps),
    )
    if tmin >= steps:
        success += 1

    scores.append(fitness_one_seed(params, seed=base_seed,
steps=steps))
    min_c.append(min(pop["carrot"]))
    min_r.append(min(pop["rabbit"]))
    min_e.append(min(pop["eagle"]))
    vol_c.append(mean_abs_change(pop["carrot"]))
    vol_r.append(mean_abs_change(pop["rabbit"]))
    vol_e.append(mean_abs_change(pop["eagle"]))

return {
    "success_rate": success / float(seeds),
    "median_fitness": float(np.median(scores)),
    "median_min_carrot": float(np.median(min_c)),
    "median_min_rabbit": float(np.median(min_r)),
    "median_min_eagle": float(np.median(min_e)),
    "median_vol_carrot": float(np.median(vol_c)),
    "median_vol_rabbit": float(np.median(vol_r)),
    "median_vol_eagle": float(np.median(vol_e)),
}

def run_baseline(theta0: np.ndarray, seed: int = 1001, steps: int =
SIM_STEPS) -> Tuple[DecodedParams, Dict[str, int]]:
    params = decode(theta0)
    env = GridWorld(
        L=params.L, p_obs=params.p_obs,
        N_e0=params.N_e0, N_r0=params.N_r0, N_c0=params.N_c0,

```

```

        rho_e=params.rho_e, rho_r=params.rho_r,
        seed=seed
    )
    pop = env.run(steps=steps, early_stop=True)
    final = {"carrot": pop["carrot"][-1], "rabbit": pop["rabbit"][-1],
"eagle": pop["eagle"][-1]}
    return params, final

def simulate_frames(theta_real: np.ndarray, seed: int = 4242, steps:
int = SIM_STEPS):
    params = decode(theta_real)
    env = GridWorld(
        L=params.L, p_obs=params.p_obs,
        N_e0=params.N_e0, N_r0=params.N_r0, N_c0=params.N_c0,
        rho_e=params.rho_e, rho_r=params.rho_r,
        seed=seed
    )

    frames = []
    for t in range(int(steps)):
        carrots = np.array([[y, x] for (x, y) in env.carrots],
dtype=float) # [col,row]
        rabbits = np.array([[a.y, a.x] for a in env.rabbits],
dtype=float)
        eagles = np.array([[a.y, a.x] for a in env.eagles],
dtype=float)

        frames.append({
            "t": t,
            "carrots": carrots,
            "rabbits": rabbits,
            "eagles": eagles,
            "n_carrot": len(env.carrots),
            "n_rabbit": len(env.rabbits),
            "n_eagle": len(env.eagles),
        })
        env.step()

    obs = np.argwhere(env.obs) # rows, cols
    obs_xy = np.array([[c, r] for (r, c) in obs], dtype=float) #
[col,row]
    return params, env.L, obs_xy, frames

def render_video(params_star: DecodedParams, W: int, obs_xy:
np.ndarray, frames: List[Dict], out_name: str =
"ecosystem_es_demo.mp4"):
    CARROT_SIZE = 30
    RABBIT_SIZE = 40
    EAGLE_SIZE = 55
    OBSTACLE_SIZE = 55

```

```

INTERVAL_MS = 30
fps = max(1, int(SIM_STEPS // max(1, INTERVAL_MS)))

fig = plt.figure(figsize=(16.8, 16.8), dpi=95)
fig.patch.set_facecolor("white")

gs = gridspec.GridSpec(
    4, 1, figure=fig,
    height_ratios=[0.30, 0.78, 0.56, 0.06],
    hspace=0.08
)

ax_leg = fig.add_subplot(gs[0, 0])
ax_anim = fig.add_subplot(gs[1, 0])
ax_hud = fig.add_subplot(gs[2, 0])
ax_prog = fig.add_subplot(gs[3, 0])

ax_anim.set_title("Ecosystem ES – Optimal Parameters Simulation",
pad=10)
ax_anim.set_xticks([])
ax_anim.set_yticks([])
ax_anim.set_facecolor("white")
ax_anim.set_xlim(-0.5, W - 0.5)
ax_anim.set_ylim(W - 0.5, -0.5)
ax_anim.set_aspect("auto")

st0 = frames[0]

carrot_scatter = ax_anim.scatter(
    st0["carrots"][:, 0] if st0["carrots"].size else [],
    st0["carrots"][:, 1] if st0["carrots"].size else [],
    s=CARROT_SIZE, c="#34C759", marker="^",
    edgecolors="k", linewidths=0.5, zorder=6, alpha=0.95
)

rabbit_scatter = ax_anim.scatter(
    st0["rabbits"][:, 0] if st0["rabbits"].size else [],
    st0["rabbits"][:, 1] if st0["rabbits"].size else [],
    s=RABBIT_SIZE, c="#FF9500", marker="o",
    edgecolors="k", linewidths=0.35, zorder=8, alpha=0.95
)

eagle_scatter = ax_anim.scatter(
    st0["eagles"][:, 0] if st0["eagles"].size else [],
    st0["eagles"][:, 1] if st0["eagles"].size else [],
    s=EAGLE_SIZE, c="#AF52DE", marker="X",
    edgecolors="k", linewidths=0.35, zorder=9, alpha=0.95
)

obs_scatter = None
if obs_xy.size:

```

```

        obs_scatter = ax_anim.scatter(
            obs_xy[:, 0], obs_xy[:, 1],
            s=OBSTACLE_SIZE, c="black", marker="s",
            linewidths=0.0, zorder=20, alpha=1.0
        )

    ax_leg.set_axis_off()
    legend_handles = [
        mlines.Line2D([], [], color="#34C759", marker="^",
            linestyle="None", markersize=12, label="Carrot"),
        mlines.Line2D([], [], color="#FF9500", marker="o",
            linestyle="None", markersize=10, label="Rabbit"),
        mlines.Line2D([], [], color="#AF52DE", marker="X",
            linestyle="None", markersize=10, label="Eagle"),
        mlines.Line2D([], [], color="black", marker="s",
            linestyle="None", markersize=10, label="Obstacle"),
    ]
    ax_leg.legend(handles=legend_handles, loc="upper left",
        framealpha=0.95, title="Legend",
        borderaxespad=0.0, handletextpad=0.6)

    ax_hud.set_axis_off()
    hud_text = ax_hud.text(
        0.0, 1.0, "",
        ha="left", va="top",
        fontsize=11, color="black",
        transform=ax_hud.transAxes,
        bbox=dict(facecolor="white", alpha=0.95,
            boxstyle="round,pad=0.40")
    )

    ax_prog.set_xlim(0, 1)
    ax_prog.set_ylim(0, 1)
    ax_prog.set_xticks([])
    ax_prog.set_yticks([])
    ax_prog.set_frame_on(True)
    ax_prog.set_title("Progress", fontsize=10, pad=2)

    progress_bg = Rectangle((0, 0), 1.0, 1.0, color="#f0f0f0")
    progress_bar = Rectangle((0, 0), 0.0, 1.0, color="#3A86FF")
    ax_prog.add_patch(progress_bg)
    ax_prog.add_patch(progress_bar)

    def draw_frame(i: int):
        st = frames[i]

        carrot_scatter.set_offsets(st["carrots"] if st["carrots"].size
    else np.empty((0, 2)))
        rabbit_scatter.set_offsets(st["rabbits"] if st["rabbits"].size
    else np.empty((0, 2)))

```

```

        eagle_scatter.set_offsets(st["eagles"] if st["eagles"].size
else np.empty((0, 2)))

    progress = (i + 1) / max(1, len(frames))
    progress_bar.set_width(progress)

    hud_text.set_text(
        "Step: {}/{}\n"
        "Parameters\n"
        "  L={} | p_obs={:.4f}\n"
        "  N_e0={} | N_r0={} | N_c0={}\n"
        "  rho_e={:.3f} | rho_r={:.3f}\n"
        "Populations\n"
        "  Carrots: {}\n"
        "  Rabbits: {}\n"
        "  Eagles: {}".format(
            i + 1, len(frames),
            params_star.L, params_star.p_obs,
            params_star.N_e0, params_star.N_r0, params_star.N_c0,
            params_star.rho_e, params_star.rho_r,
            st["n_carrot"], st["n_rabbit"], st["n_eagle"]
        )
    )

    artists = [carrot_scatter, rabbit_scatter, eagle_scatter,
hud_text, progress_bar]
    if obs_scatter is not None:
        artists.append(obs_scatter)
    return tuple(artists)

    anim = FuncAnimation(fig, draw_frame, frames=len(frames),
interval=INTERVAL_MS, blit=False, repeat=False)
    writer = FFMpegWriter(
        fps=fps,
        codec="libx264",
        bitrate=2400,
        extra_args=["-pix_fmt", "yuv420p", "-movflags", "+faststart"]
    )
    anim.save(out_name, writer=writer, dpi=95,
savefig_kwargs={"facecolor": "white"})
    plt.close(fig)
    print(f"Saved: {out_name}")
    display(Video(out_name, embed=True))

#
=====
=====
# RUN (will work after TODOs are completed)
#

```

```

=====
=====
theta0 = np.array([50.0, 0.075, 6.0, 60.0, 500.0, 0.50, 0.50],
dtype=float)

base_params, base_final = run_baseline(theta0, seed=1001,
steps=SIM_STEPS)
print(f"Baseline decoded: {base_params}")
print(f"Baseline final: {base_final}")

# TODO: after implementing run_es(...), uncomment and run:
# best_theta, best_fit, history = run_es(
#     generations=60,
#     mu=10,
#     lam=40,
#     seed=7,
#     S=5,
#     steps=SIM_STEPS,
#     verbose_every=1
# )
# print(f"\nBest fitness: {best_fit}")
# print(f"Best decoded: {decode(best_theta)}")
# report = validate_solution(best_theta, seeds=20, steps=SIM_STEPS)
# print(f"\nValidation report: {report}")
# params_star, W, obs_xy, frames = simulate_frames(best_theta,
seed=4242, steps=SIM_STEPS)
# render_video(params_star, W, obs_xy, frames,
out_name="ecosystem_es_demo.mp4")

```

Ex.2 Ecosystem Stability Optimisation with a Genetic Algorithm (TODO completion)

A 2D grid world contains three agent types: carrots (resource), rabbits (herbivores), and eagles (predators). The grid is square with side length L cells. The world includes static obstacles that block both movement and line-of-sight. The simulation is stochastic due to random reproduction placement and random tie-breaking in movement decisions.

The optimisation goal is to identify initial population sizes, demographic ratios, and environment configuration that yield a stable, persistent ecosystem over **100 simulation steps**.

Optimisation is performed using a **Genetic Algorithm (GA)** operating on a **mixed discrete–continuous** parameter vector.

Provided notebook scaffold

A complete simulator and evaluation pipeline are provided. Several GA components are intentionally incomplete and marked with `# TODO / NotImplementedError`.

Your task is to complete the `TODO` sections so that the notebook can:

1. Run a baseline simulation,
 2. Run the GA for multiple generations and print per-generation logs,
 3. Validate the best genome on 20 seeds,
 4. Render a 100-step MP4 video of the best solution.
-

How to approach the TODOs (practical tips)

Complete and test the TODOs in the following order:

1. `tournament_select(...)`
Start here because it is small and easy to verify.
2. `mutate(...)`
Confirm bounds are enforced via `repair_inplace(...)`.
3. `crossover(...)`
Implement mixed-type crossover: swap integers, blend reals.
4. `run_ga(...)`
Implement the GA loop last. Use the provided fitness (`robust_eval`) and dominance comparator (`compare_stats`) exactly as given.

Recommended debugging strategy:

- Run with small settings first (e.g., `generations=3`, `pop_size=10`, `S=2`).
 - Print decoded genomes occasionally (`decode(...)`) to confirm constraints.
 - Confirm that `feas_rate` changes over time; if it stays at 0, consider adjusting bounds or penalties after correctness is confirmed.
-

World and agents

Map and obstacles

The environment is an `L × L` grid where `L` is an integer in `[25, 40]`.

Obstacles occupy a fraction `p_obs` of the grid, with `p_obs` in `[0.05, 0.10]`.

Obstacles:

- block agent movement;
- block eagle line-of-sight.

Vision occlusion rule:

An eagle can detect rabbits only along unobstructed line-of-sight rays. Rabbits behind obstacles are invisible. Occlusion is computed geometrically.

Agent states (rabbits and eagles)

Each rabbit and eagle has:

- age (juvenile or adult),
- sex (male or female),
- energy.

Ageing: juveniles become adults after **5** simulation steps.

Vision:

- Rabbits can see up to **3** cells.
- Eagles can see up to **5** cells.

Starvation:

- Energy decreases by **1** each step.
- Agents die when energy reaches **0**.

Feeding:

- Rabbits gain energy by eating carrots in their cell.
 - Eagles gain energy by catching a rabbit in their cell.
-

Movement

Movement is blocked by obstacles and grid boundaries.

Behaviour:

- Rabbits move toward nearby carrots if detected; otherwise perform a random walk.
- Eagles move toward the nearest visible rabbit; if none are visible, perform a random walk.

Movement tie-breaking is stochastic but reproducible using a seeded random number generator.

Note: movement speeds are defined as constants in the provided code (do not alter unless instructed).

Reproduction

Reproduction is energy-gated.

Rules:

- Only adults can reproduce.
- Reproduction occurs when an adult male and adult female of the same species occupy the same cell.

- Both parents pay an energy cost.
- Offspring is spawned into a random empty neighbouring cell (Moore neighbourhood, radius 1).
- If no empty neighbouring cell exists, reproduction fails.

Carrots reproduce asexually with a fixed per-step probability r_c and spawn into a random empty neighbouring cell (radius 1).
Carrot reproduction is not optimised.

Optimisation decision variables (what the GA controls)

The GA optimises a genome θ with the following genes:

- L : grid size, integer in $[25, 40]$
- p_{obs} : obstacle fraction, real in $[0.05, 0.10]$
- N_{e0} : initial eagles, integer in $[MIN_EAGLES, MAX_EAGLES]$
- N_{r0} : initial rabbits, integer in $[MIN_RABBITS, MAX_RABBITS]$
- N_{c0} : initial carrots, integer in $[MIN_CARROTS, MAX_CARROTS]$
- p_e : female ratio (eagles), real in $[0.40, 0.60]$
- p_r : female ratio (rabbits), real in $[0.40, 0.60]$

Decoding clamps values to bounds and rounds integer variables.

Objective and constraints

Hard constraint:

No species may go extinct during the first **100** simulation steps.

Graded fitness signal (robust under noise)

Each candidate θ is evaluated across S random seeds (e.g., $S = 5$). For each seed, the simulation runs for 100 steps and records:

- extinction times,
- minimum populations,
- population volatility (mean absolute change per step),
- overcrowding penalty.

Per-seed fitness:

If any species goes extinct before step 100:

$$\text{fitness_seed} = -M \times (100 - \min(T_{\text{ext}}))$$

Otherwise:

```
fitness_seed = w1 × min(N_min) - w2 × volatility - w3 ×  
overcrowding_penalty
```

Robust aggregation:

```
fitness(θ) = median(fitness_seed across S seeds)
```

Feasibility-first dominance is applied in selection: any feasible genome is always preferred over any infeasible one.

Genetic Algorithm design

The GA scaffold uses:

- Mixed-type genome stored as floats, decoded into ints/reals
 - Feasibility-first tournament selection
 - Crossover:
 - uniform swap for integer genes
 - BLX- α (blend) for real-valued genes
 - Mutation:
 - bounded integer step for integer genes
 - Gaussian mutation for real genes
 - Elitism: top fraction copied unchanged
 - Elite re-evaluation on fresh seeds (optional toggle)
-

Student tasks: complete the TODOs

Complete the functions marked as TODO in the provided GA notebook:

1. `crossover(parent1, parent2, rng, ...)`
2. `mutate(genome, rng, ...)`
3. `tournament_select(pop, stats, rng, k=...)`
4. `run_ga(...)`

After completing the TODOs, run the final “RUN” section to execute:

- Baseline → GA → Validation → Render video

Expected outputs (once complete):

- Baseline decoded + baseline final populations
 - Per-generation GA log lines (feasibility rate, median score, best score, decoded best)
 - Validation report on 20 seeds
 - Saved MP4 file (rendered simulation)
-

Submission checklist

- Code runs without `NotImplementedError`.
- GA prints generation logs and returns a best genome.
- Validation executes and prints a report dictionary.
- Video render runs and saves the MP4.

```
#
=====
=====
# Self-contained GA + Simulator + Video Render (Rabbits-Carrots-Eagles)
# Copy into a fresh notebook and run top-to-bottom.
#
# Student version: GA core contains TODOs to complete.
#
# Includes:
# - GridWorld simulator with obstacles and eagle line-of-sight occlusion
# - Robust GA skeleton (feasibility-first, median across seeds, elite re-evaluation)
# - simulate_frames(...) and render_video(...) in the provided template style
# - Baseline -> GA -> Validation -> Render 100-step MP4
#
# Dependencies: numpy, matplotlib, IPython (for Video display), ffmpeg available
#
=====
=====

from __future__ import annotations

from dataclasses import dataclass
from typing import Dict, List, Tuple, Optional
import numpy as np

import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import matplotlib.lines as mlines
from matplotlib.patches import Rectangle
from matplotlib.animation import FuncAnimation, FFMpegWriter
from IPython.display import Video, display

#
=====
=====
# USER-CONFIGURABLE CONSTANTS
#
=====
```

```

=====

SIM_STEPS = 100
S_SEEDS = 5

# Decision variable bounds (GA)
MIN_L, MAX_L = 25, 40
MIN_POBS, MAX_POBS = 0.05, 0.10

MIN_EAGLES, MAX_EAGLES = 2, 4
MIN_RABBITS, MAX_RABBITS = 100, 200
MIN_CARROTS, MAX_CARROTS = 300, 400

MIN_RHO, MAX_RHO = 0.40, 0.60

# Simulator constants
R_C = 0.05

RABBIT_VISION = 3
EAGLE_VISION = 5
RABBIT_SPEED = 2          # Chebyshev steps
EAGLE_SPEED = 3

JUVENILE_STEPS = 5
REPRO_COST = 3

RABBIT_ENERGY_INIT = 8
EAGLE_ENERGY_INIT = 10
RABBIT_ENERGY_CAP = 12
EAGLE_ENERGY_CAP = 16
RABBIT_EAT_GAIN = 6
EAGLE_EAT_GAIN = 8

# Fitness constants
M_PENALTY = 50.0
W1 = 1.0
W2 = 1.0
W3 = 1.0

#
=====
=====
# UTILITIES
#
=====
=====

def _clamp(x: float, lo: float, hi: float) -> float:
    return float(min(max(float(x), float(lo)), float(hi)))

```

```

def seed_set(base_seed: int, S: int) -> List[int]:
    return [int(base_seed + SIM_STEPS * i) for i in range(int(S))]

def bresenham_line(x0: int, y0: int, x1: int, y1: int) ->
List[Tuple[int, int]]:
    cells: List[Tuple[int, int]] = []
    dx = abs(x1 - x0)
    dy = abs(y1 - y0)
    sx = 1 if x0 < x1 else -1
    sy = 1 if y0 < y1 else -1
    err = dx - dy
    x, y = x0, y0
    while True:
        cells.append((x, y))
        if x == x1 and y == y1:
            break
        e2 = 2 * err
        if e2 > -dy:
            err -= dy
            x += sx
        if e2 < dx:
            err += dx
            y += sy
    return cells

def extinction_time(series: List[int], max_steps: int = SIM_STEPS) ->
int:
    for t, n in enumerate(series):
        if n <= 0:
            return t
    return max_steps

def mean_abs_change(series: List[int]) -> float:
    if len(series) < 2:
        return 0.0
    diffs = [abs(series[i + 1] - series[i]) for i in range(len(series)
- 1)]
    return float(np.mean(diffs))

def overcrowding_penalty(pop_hist: Dict[str, List[int]], L: int,
p_obs: float) -> float:
    free = int(round(L * L * (1.0 - p_obs)))
    if free <= 0:
        return 1e6
    total = np.array(pop_hist["carrot"]) +
np.array(pop_hist["rabbit"]) + np.array(pop_hist["eagle"])
    ratio = float(np.max(total / free)) if len(total) else 0.0
    return float(max(0.0, ratio - 0.80) * 100.0)

```

```

#
=====

=====
# SIMULATOR
#
=====

=====

@dataclass
class Agent:
    x: int
    y: int
    species: str # "rabbit" or "eagle"
    sex: int      # 0 male, 1 female
    age_steps: int
    energy: int

    @property
    def adult(self) -> bool:
        return self.age_steps >= JUVENILE_STEPS

@dataclass(frozen=True)
class DecodedParams:
    L: int
    p_obs: float
    N_e0: int
    N_r0: int
    N_c0: int
    rho_e: float
    rho_r: float

class GridWorld:
    _NBR8 = [(-1, -1), (-1, 0), (-1, 1),
              (0, -1),  (0, 0),  (0, 1),
              (1, -1),  (1, 0),  (1, 1)]

    def __init__(
        self,
        L: int,
        p_obs: float,
        N_e0: int,
        N_r0: int,
        N_c0: int,
        rho_e: float,
        rho_r: float,
        seed: int,
        r_c: float = R_C,
    ):

```

```

self.L = int(L)
self.rng = np.random.default_rng(int(seed))
self.r_c = float(r_c)

self.obs = np.zeros((self.L, self.L), dtype=bool)
self._place_obstacles(p_obs)

self.carrots: set[Tuple[int, int]] = set()
self.rabbits: List[Agent] = []
self.eagles: List[Agent] = []

self.pop_history: Dict[str, List[int]] = {"carrot": [],
"rabbit": [], "eagle": []}

self._spawn_initial(N_c0, N_r0, N_e0, rho_r, rho_e)

def _place_obstacles(self, p_obs: float) -> None:
    total = self.L * self.L
    n_obs = int(round(float(p_obs) * total))
    idx = self.rng.choice(total, size=n_obs, replace=False)
    xs = idx // self.L
    ys = idx % self.L
    self.obs[xs, ys] = True

def in_bounds(self, x: int, y: int) -> bool:
    return 0 <= x < self.L and 0 <= y < self.L

def is_blocked(self, x: int, y: int) -> bool:
    return bool(self.obs[x, y])

def _occupied_by_any_agent(self, x: int, y: int) -> bool:
    for a in self.rabbits:
        if a.x == x and a.y == y:
            return True
    for a in self.eagles:
        if a.x == x and a.y == y:
            return True
    return False

def is_empty(self, x: int, y: int) -> bool:
    if not self.in_bounds(x, y) or self.is_blocked(x, y):
        return False
    if (x, y) in self.carrots:
        return False
    if self._occupied_by_any_agent(x, y):
        return False
    return True

def _random_empty_cell(self) -> Tuple[int, int]:
    for _ in range(200000):

```

```

        x = int(self.rng.integers(0, self.L))
        y = int(self.rng.integers(0, self.L))
        if self.is_empty(x, y):
            return (x, y)
        raise RuntimeError("Failed to sample empty cell. Reduce
densities/obstacles or increase L.")

    def _spawn_initial(self, N_c0: int, N_r0: int, N_e0: int, rho_r:
float, rho_e: float) -> None:
        for _ in range(int(N_c0)):
            x, y = self._random_empty_cell()
            self.carrots.add((x, y))

        n_rf = int(round(int(N_r0) * float(rho_r)))
        n_rm = int(N_r0) - n_rf
        r_sexes = [1] * n_rf + [0] * n_rm
        self.rng.shuffle(r_sexes)
        for s in r_sexes:
            x, y = self._random_empty_cell()
            self.rabbits.append(Agent(x, y, "rabbit", int(s), 0,
RABBIT_ENERGY_INIT))

        n_ef = int(round(int(N_e0) * float(rho_e)))
        n_em = int(N_e0) - n_ef
        e_sexes = [1] * n_ef + [0] * n_em
        self.rng.shuffle(e_sexes)
        for s in e_sexes:
            x, y = self._random_empty_cell()
            self.eagles.append(Agent(x, y, "eagle", int(s), 0,
EAGLE_ENERGY_INIT))

    def _reachable_cells(self, start: Tuple[int, int], max_steps: int)
-> List[Tuple[int, int]]:
        sx, sy = start
        q = [(sx, sy, 0)]
        seen = {(sx, sy)}
        out = [(sx, sy)]
        while q:
            x, y, d = q.pop(0)
            if d == max_steps:
                continue
            for dx, dy in self._NBR8:
                nx, ny = x + dx, y + dy
                if not self.in_bounds(nx, ny) or self.is_blocked(nx,
ny):
                    continue
                if (nx, ny) in seen:
                    continue
                seen.add((nx, ny))
                out.append((nx, ny))

```

```

        q.append((nx, ny, d + 1))
    return out

def _shortest_path_step(self, start: Tuple[int, int], goal:
Tuple[int, int], max_steps: int) -> Tuple[int, int]:
    if start == goal:
        return start

    sx, sy = start
    gx, gy = goal

    q = [(sx, sy)]
    parent: Dict[Tuple[int, int], Tuple[int, int]] = {}
    seen = {(sx, sy)}

    while q:
        x, y = q.pop(0)
        if (x, y) == (gx, gy):
            break
        for dx, dy in self._NBR8:
            nx, ny = x + dx, y + dy
            if not self.in_bounds(nx, ny) or self.is_blocked(nx,
ny):
                continue
            if (nx, ny) in seen:
                continue
            seen.add((nx, ny))
            parent[(nx, ny)] = (x, y)
            q.append((nx, ny))

    if (gx, gy) not in seen:
        reach = self._reachable_cells(start, max_steps)
        reach = [p for p in reach if p != start] or reach
        return reach[int(self.rng.integers(0, len(reach)))]

    path = [(gx, gy)]
    cur = (gx, gy)
    while cur != (sx, sy):
        cur = parent[cur]
        path.append(cur)
    path.reverse()
    idx = min(max_steps, len(path) - 1)
    return path[idx]

def _nearest_carrot_within(self, x: int, y: int, radius: int) ->
Optional[Tuple[int, int]]:
    best = None
    best_d = 10**9
    for (cx, cy) in self.carrots:
        d = max(abs(cx - x), abs(cy - y))

```

```

        if d <= radius:
            if d < best_d or (d == best_d and self.rng.random() <
0.5):
                best_d = d
                best = (cx, cy)
        return best

def _eagle_visible_rabbits(self, ex: int, ey: int, radius: int) ->
List[Tuple[int, int]]:
    out: List[Tuple[int, int]] = []
    for r in self.rabbits:
        if max(abs(r.x - ex), abs(r.y - ey)) > radius:
            continue
        line = bresenham_line(ex, ey, r.x, r.y)
        blocked = False
        for (lx, ly) in line[1:-1]:
            if self.is_blocked(lx, ly):
                blocked = True
                break
        if not blocked:
            out.append((r.x, r.y))
    return out

def _nearest_visible_rabbit(self, ex: int, ey: int, radius: int) -
> Optional[Tuple[int, int]]:
    vis = self._eagle_visible_rabbits(ex, ey, radius)
    if not vis:
        return None
    best = None
    best_d = 10**9
    for (rx, ry) in vis:
        d = max(abs(rx - ex), abs(ry - ey))
        if d < best_d or (d == best_d and self.rng.random() <
0.5):
            best_d = d
            best = (rx, ry)
    return best

def _record_pops(self) -> None:
    self.pop_history["carrot"].append(len(self.carrots))
    self.pop_history["rabbit"].append(len(self.rabbits))
    self.pop_history["eagle"].append(len(self.eagles))

def _carrot_repro(self) -> None:
    if not self.carrots:
        return
    new_positions: List[Tuple[int, int]] = []
    for (x, y) in list(self.carrots):
        if self.rng.random() < self.r_c:
            candidates = []

```

```

        for dx, dy in self._NBR8:
            nx, ny = x + dx, y + dy
            if self.is_empty(nx, ny):
                candidates.append((nx, ny))
        if candidates:
            new_positions.append(candidates[int(self.rng.integers(0,
len(candidates)))]])
            for pos in new_positions:
                self.carrots.add(pos)

        def _move_rabbits(self) -> None:
            for r in self.rabbits:
                target = self._nearest_carrot_within(r.x, r.y,
RABBIT_VISION)
                if target is None:
                    reach = self._reachable_cells((r.x, r.y),
RABBIT_SPEED)
                    reach = [p for p in reach if p != (r.x, r.y)] or reach
                    r.x, r.y = reach[int(self.rng.integers(0,
len(reach)))]
                else:
                    r.x, r.y = self._shortest_path_step((r.x, r.y),
target, RABBIT_SPEED)

        def _move_eagles(self) -> None:
            for e in self.eagles:
                target = self._nearest_visible_rabbit(e.x, e.y,
EAGLE_VISION)
                if target is None:
                    reach = self._reachable_cells((e.x, e.y), EAGLE_SPEED)
                    reach = [p for p in reach if p != (e.x, e.y)] or reach
                    e.x, e.y = reach[int(self.rng.integers(0,
len(reach)))]
                else:
                    e.x, e.y = self._shortest_path_step((e.x, e.y),
target, EAGLE_SPEED)

        def _interactions(self) -> None:
            for r in self.rabbits:
                pos = (r.x, r.y)
                if pos in self.carrots:
                    self.carrots.remove(pos)
                    r.energy = min(RABBIT_ENERGY_CAP, r.energy +
RABBIT_EAT_GAIN)

            if not self.rabbits or not self.eagles:
                return

            rabbits_by_cell: Dict[Tuple[int, int], List[int]] = {}

```

```

        for i, r in enumerate(self.rabbits):
            rabbits_by_cell.setdefault((r.x, r.y), []).append(i)

        dead = set()
        for e in self.eagles:
            cell = (e.x, e.y)
            if cell in rabbits_by_cell:
                alive = [i for i in rabbits_by_cell[cell] if i not in
dead]
                    if alive:
                        victim = alive[int(self.rng.integers(0,
len(alive)))]
                        dead.add(victim)
                        e.energy = min(EAGLE_ENERGY_CAP, e.energy +
EAGLE_EAT_GAIN)

                if dead:
                    self.rabbits = [r for i, r in enumerate(self.rabbits) if i
not in dead]

        def _reproduce_species(self, agents: List[Agent], species: str) ->
None:
            if not agents:
                return

            by_cell: Dict[Tuple[int, int], List[int]] = {}
            for i, a in enumerate(agents):
                by_cell.setdefault((a.x, a.y), []).append(i)

            newborns: List[Agent] = []
            for cell, idxs in by_cell.items():
                males = [i for i in idxs if agents[i].adult and
agents[i].sex == 0 and agents[i].energy >= REPRO_COST]
                females = [i for i in idxs if agents[i].adult and
agents[i].sex == 1 and agents[i].energy >= REPRO_COST]
                if not males or not females:
                    continue

                self.rng.shuffle(males)
                self.rng.shuffle(females)
                n_pairs = min(len(males), len(females))

                for k in range(n_pairs):
                    mi = males[k]
                    fi = females[k]
                    agents[mi].energy -= REPRO_COST
                    agents[fi].energy -= REPRO_COST

                    x, y = cell
                    candidates = []

```

```

        for dx, dy in self._NBR8:
            nx, ny = x + dx, y + dy
            if not self.in_bounds(nx, ny) or
self.is_blocked(nx, ny):
                continue
            if (nx, ny) in self.carrots:
                continue
            if self._occupied_by_any_agent(nx, ny):
                continue
            candidates.append((nx, ny))
            if not candidates:
                continue

            bx, by = candidates[int(self.rng.integers(0,
len(candidates)))]
            sex = int(self.rng.integers(0, 2))
            init_e = RABBIT_ENERGY_INIT if species == "rabbit"
else EAGLE_ENERGY_INIT
            newborns.append(Agent(bx, by, species, sex, 0,
init_e))

        agents.extend(newborns)

    def _reproduction(self) -> None:
        self._reproduce_species(self.rabbits, "rabbit")
        self._reproduce_species(self.eagles, "eagle")

    def _age_and_decay(self) -> None:
        for r in self.rabbits:
            r.age_steps += 1
            r.energy -= 1
        for e in self.eagles:
            e.age_steps += 1
            e.energy -= 1

    def _cull_dead(self) -> None:
        self.rabbits = [r for r in self.rabbits if r.energy > 0]
        self.eagles = [e for e in self.eagles if e.energy > 0]

    def step(self) -> None:
        self._record_pops()
        self._carrot_repro()
        self._move_rabbits()
        self._move_eagles()
        self._interactions()
        self._reproduction()
        self._age_and_decay()
        self._cull_dead()

    def run(self, steps: int = SIM_STEPS, early_stop: bool = True) ->

```

```

Dict[str, List[int]]:
    for _ in range(int(steps)):
        self.step()
        if early_stop and (len(self.carrots) == 0 or
len(self.rabbits) == 0 or len(self.eagles) == 0):
            self._record_pops()
            break
    return self.pop_history

#
=====
=====
# DECODING (GA genome -> parameters)
#
=====
=====

# Genome layout: [L, p_obs, N_e0, N_r0, N_c0, rho_e, rho_r]
DIM = 7
INT_IDXS = {0, 2, 3, 4}
REAL_IDXS = {1, 5, 6}

BOUNDS = np.array([
    [MIN_L,      MAX_L],
    [MIN_POBS,   MAX_POBS],
    [MIN_EAGLES, MAX_EAGLES],
    [MIN_RABBITS, MAX_RABBITS],
    [MIN_CARROTS, MAX_CARROTS],
    [MIN_RHO,    MAX_RHO],
    [MIN_RHO,    MAX_RHO],
], dtype=float)

def decode(theta_real: np.ndarray) -> DecodedParams:
    v = np.asarray(theta_real, dtype=float).copy()
    L = int(round(_clamp(v[0], *BOUNDS[0])))
    p_obs = _clamp(v[1], *BOUNDS[1])
    N_e0 = int(round(_clamp(v[2], *BOUNDS[2])))
    N_r0 = int(round(_clamp(v[3], *BOUNDS[3])))
    N_c0 = int(round(_clamp(v[4], *BOUNDS[4])))
    rho_e = _clamp(v[5], *BOUNDS[5])
    rho_r = _clamp(v[6], *BOUNDS[6])
    return DecodedParams(L=L, p_obs=p_obs, N_e0=N_e0, N_r0=N_r0,
N_c0=N_c0, rho_e=rho_e, rho_r=rho_r)

def repair_inplace(g: np.ndarray) -> None:
    for i in range(DIM):
        g[i] = _clamp(g[i], BOUNDS[i, 0], BOUNDS[i, 1])

```

```

#
=====
=====
# FITNESS (robust median over S seeds, feasibility-first)
#
=====
=====

def eval_one_seed(params: DecodedParams, seed: int, steps: int =
SIM_STEPS) -> Dict[str, float]:
    env = GridWorld(
        L=params.L, p_obs=params.p_obs,
        N_e0=params.N_e0, N_r0=params.N_r0, N_c0=params.N_c0,
        rho_e=params.rho_e, rho_r=params.rho_r,
        seed=seed, r_c=R_C
    )
    pop = env.run(steps=steps, early_stop=True)

    t_c = extinction_time(pop["carrot"], steps)
    t_r = extinction_time(pop["rabbit"], steps)
    t_e = extinction_time(pop["eagle"], steps)
    t_min = min(t_c, t_r, t_e)

    feasible = float(t_min >= steps)

    if feasible < 0.5:
        score = -float(M_PENALTY) * float(steps - t_min)
    else:
        nmin = float(min(min(pop["carrot"]), min(pop["rabbit"]),
min(pop["eagle"])))
        vol = float(mean_abs_change(pop["carrot"]) +
mean_abs_change(pop["rabbit"]) + mean_abs_change(pop["eagle"]))
        over = float(overcrowding_penalty(pop, params.L,
params.p_obs))
        score = float(W1 * nmin - W2 * vol - W3 * over)

    return {"score": float(score), "feasible": float(feasible)}

def robust_eval(theta: np.ndarray, seeds: List[int], steps: int =
SIM_STEPS) -> Dict[str, float]:
    params = decode(theta)
    per = [eval_one_seed(params, s, steps=steps) for s in seeds]
    scores = np.array([d["score"] for d in per], dtype=float)
    feas = np.array([d["feasible"] for d in per], dtype=float)
    return {
        "feasible": float(np.all(feas > 0.5)),
        "score": float(np.median(scores)),
    }

```

```

#
=====
=====
# GA OPERATORS (some parts are TODO)
#
=====
=====

def random_genome(rng: np.random.Generator) -> np.ndarray:
    # Provided: uniform initialisation in the decision bounds.
    return rng.uniform(BOUNDS[:, 0], BOUNDS[:, 1]).astype(float)

def crossover(parent1: np.ndarray, parent2: np.ndarray, rng:
np.random.Generator,
                p_swap_int: float = 0.5, alpha_blend: float = 0.25) ->
np.ndarray:
    # TODO:
    # Implement crossover that mixes integer-like genes differently
    from real-like genes.
    # Recommended approach (as in lectures):
    # - For INT_IDXS: uniform swap per gene with probability
    p_swap_int
    # - For REAL_IDXS: BLX-alpha blend crossover with alpha_blend
    # Requirements:
    # - Return a child genome of shape (DIM,)
    # - Call repair_inplace(child) before returning
    raise NotImplementedError

def mutate(genome: np.ndarray, rng: np.random.Generator,
            p_mut_gene: float = 0.25,
            int_step: Optional[Dict[int, int]] = None,
            real_sigma: Optional[Dict[int, float]] = None) ->
np.ndarray:
    # TODO:
    # Implement per-gene mutation.
    # Recommended approach:
    # - With probability p_mut_gene per gene:
    #   - If gene is integer-like: add an integer delta in [-step,
+step]
    #   - If gene is real-like: add Gaussian noise  $N(0, \sigma)$ 
    # Requirements:
    # - Use default dictionaries if int_step/real_sigma are None
    # - Call repair_inplace(mutant) before returning
    raise NotImplementedError

def compare_stats(a: Dict[str, float], b: Dict[str, float]) -> int:
    # Provided: feasibility-first dominance comparison.
    fa, fb = a["feasible"], b["feasible"]
    if fa > fb:
        return 1

```

```

    if fb > fa:
        return -1
    if a["score"] > b["score"]:
        return 1
    if b["score"] > a["score"]:
        return -1
    return 0

def tournament_select(pop: List[np.ndarray], stats: List[Dict[str,
float]],
                      rng: np.random.Generator, k: int = 3) ->
np.ndarray:
    # TODO:
    # Implement tournament selection of size k using
compare_stats(...).
    # Requirements:
    # - Sample k indices (with replacement is acceptable)
    # - Return the genome of the best candidate (copy not required)
    raise NotImplementedError

#
=====
=====
# GA MAIN (TODO section)
#
=====
=====

def run_ga(
    generations: int = 40,
    pop_size: int = 60,
    elite_frac: float = 0.10,
    tournament_k: int = 3,
    crossover_rate: float = 0.85,
    mutation_rate: float = 0.25,
    S: int = S_SEEDS,
    steps: int = SIM_STEPS,
    seed: int = 7,
    reevaluate_elites: bool = True,
    verbose_every: int = 1,
):
    # TODO:
    # Implement the main GA loop.
    #
    # Recommended structure:
    # 1) Initialise pop with random_genome(...)
    # 2) For each generation:
    #     - Build an evaluation seed set (seed_set(...))
    #     - Compute stats for each genome via robust_eval(...)

```

```

# - Sort population by (feasible desc, score desc)
# - Copy top n_elite into the next population
# - Optionally re-evaluate elites on fresh seeds
(reevaluate_elites)
# - Track global best genome/best_stat via compare_stats(...)
# - Print generation summary (feas_rate, median score, best
score, decoded best)
# - Fill remainder of next_pop using:
#   - tournament selection
#   - crossover with probability crossover_rate
#   - mutation
# 3) Return (best_genome, best_stat)
raise NotImplementedError

#
=====
=====
# BASELINE + VALIDATION (provided)
#
=====
=====

def run_baseline(theta0: np.ndarray, seed: int = 1001, steps: int =
SIM_STEPS) -> Tuple[DecodedParams, Dict[str, int]]:
    params = decode(theta0)
    env = GridWorld(
        L=params.L, p_obs=params.p_obs,
        N_e0=params.N_e0, N_r0=params.N_r0, N_c0=params.N_c0,
        rho_e=params.rho_e, rho_r=params.rho_r,
        seed=seed
    )
    pop = env.run(steps=steps, early_stop=True)
    final = {"carrot": pop["carrot"][-1], "rabbit": pop["rabbit"][-1],
"eagle": pop["eagle"][-1]}
    return params, final

def validate_solution(theta: np.ndarray, seeds: int = 20, steps: int =
SIM_STEPS) -> Dict[str, float]:
    params = decode(theta)
    success = 0
    scores = []
    min_c, min_r, min_e = [], [], []
    for i in range(int(seeds)):
        s = 500000 + 97 * i
        env = GridWorld(
            L=params.L, p_obs=params.p_obs,
            N_e0=params.N_e0, N_r0=params.N_r0, N_c0=params.N_c0,
            rho_e=params.rho_e, rho_r=params.rho_r,
            seed=s

```

```

    )
    pop = env.run(steps=steps, early_stop=True)
    tmin = min(extinction_time(pop["carrot"], steps),
               extinction_time(pop["rabbit"], steps),
               extinction_time(pop["eagle"], steps))
    if tmin >= steps:
        success += 1
    scores.append(eval_one_seed(params, s, steps=steps)["score"])
    min_c.append(min(pop["carrot"]))
    min_r.append(min(pop["rabbit"]))
    min_e.append(min(pop["eagle"]))
return {
    "success_rate": success / float(seeds),
    "median_fitness": float(np.median(scores)),
    "median_min_carrot": float(np.median(min_c)),
    "median_min_rabbit": float(np.median(min_r)),
    "median_min_eagle": float(np.median(min_e)),
}

#
=====
# SIMULATION FRAMES + VIDEO RENDER (template requested)
#
=====

def simulate_frames(theta_real: np.ndarray, seed: int = 4242, steps:
int = SIM_STEPS):
    params = decode(theta_real)
    env = GridWorld(
        L=params.L, p_obs=params.p_obs,
        N_e0=params.N_e0, N_r0=params.N_r0, N_c0=params.N_c0,
        rho_e=params.rho_e, rho_r=params.rho_r,
        seed=seed
    )

    frames = []
    for t in range(int(steps)):
        carrots = np.array([[y, x] for (x, y) in env.carrots],
dtype=float) # [col,row]
        rabbits = np.array([[a.y, a.x] for a in env.rabbits],
dtype=float)
        eagles = np.array([[a.y, a.x] for a in env.eagles],
dtype=float)

        frames.append({
            "t": t,
            "carrots": carrots,

```

```

        "rabbits": rabbits,
        "eagles": eagles,
        "n_carrot": len(env.carrots),
        "n_rabbit": len(env.rabbits),
        "n_eagle": len(env.eagles),
    })
    env.step()

    obs = np.argwhere(env.obs) # rows, cols
    obs_xy = np.array([[c, r] for (r, c) in obs], dtype=float) #
[col,row]
    return params, env.L, obs_xy, frames

def render_video(params_star: DecodedParams, W: int, obs_xy:
np.ndarray, frames: List[Dict], out_name: str =
"ecosystem_ga_demo.mp4"):
    CARROT_SIZE = 30
    RABBIT_SIZE = 40
    EAGLE_SIZE = 55
    OBSTACLE_SIZE = 55

    INTERVAL_MS = 30
    fps = max(1, int(SIM_STEPS // max(1, INTERVAL_MS)))

    fig = plt.figure(figsize=(16.8, 16.8), dpi=95)
    fig.patch.set_facecolor("white")

    gs = gridspec.GridSpec(
        4, 1, figure=fig,
        height_ratios=[0.30, 0.78, 0.56, 0.06],
        hspace=0.08
    )

    ax_leg = fig.add_subplot(gs[0, 0])
    ax_anim = fig.add_subplot(gs[1, 0])
    ax_hud = fig.add_subplot(gs[2, 0])
    ax_prog = fig.add_subplot(gs[3, 0])

    ax_anim.set_title("Ecosystem GA – Best Parameters Simulation",
pad=10)
    ax_anim.set_xticks([])
    ax_anim.set_yticks([])
    ax_anim.set_facecolor("white")
    ax_anim.set_xlim(-0.5, W - 0.5)
    ax_anim.set_ylim(W - 0.5, -0.5)
    ax_anim.set_aspect("auto")

    st0 = frames[0]

```

```

carrot_scatter = ax_anim.scatter(
    st0["carrots"][:, 0] if st0["carrots"].size else [],
    st0["carrots"][:, 1] if st0["carrots"].size else [],
    s=CARROT_SIZE, c="#34C759", marker="^",
    edgecolors="k", linewidths=0.5, zorder=6, alpha=0.95
)
rabbit_scatter = ax_anim.scatter(
    st0["rabbits"][:, 0] if st0["rabbits"].size else [],
    st0["rabbits"][:, 1] if st0["rabbits"].size else [],
    s=RABBIT_SIZE, c="#FF9500", marker="o",
    edgecolors="k", linewidths=0.35, zorder=8, alpha=0.95
)
eagle_scatter = ax_anim.scatter(
    st0["eagles"][:, 0] if st0["eagles"].size else [],
    st0["eagles"][:, 1] if st0["eagles"].size else [],
    s=EAGLE_SIZE, c="#AF52DE", marker="X",
    edgecolors="k", linewidths=0.35, zorder=9, alpha=0.95
)

obs_scatter = None
if obs_xy.size:
    obs_scatter = ax_anim.scatter(
        obs_xy[:, 0], obs_xy[:, 1],
        s=OBSTACLE_SIZE, c="black", marker="s",
        linewidths=0.0, zorder=20, alpha=1.0
    )

ax_leg.set_axis_off()
legend_handles = [
    mlines.Line2D([], [], color="#34C759", marker="^",
linestyle="None", markersize=12, label="Carrot"),
    mlines.Line2D([], [], color="#FF9500", marker="o",
linestyle="None", markersize=10, label="Rabbit"),
    mlines.Line2D([], [], color="#AF52DE", marker="X",
linestyle="None", markersize=10, label="Eagle"),
    mlines.Line2D([], [], color="black", marker="s",
linestyle="None", markersize=10, label="Obstacle"),
]
ax_leg.legend(handles=legend_handles, loc="upper left",
              framealpha=0.95, title="Legend",
              borderaxespad=0.0, handletextpad=0.6)

ax_hud.set_axis_off()
hud_text = ax_hud.text(
    0.0, 1.0, "",
    ha="left", va="top",
    fontsize=11, color="black",
    transform=ax_hud.transAxes,
    bbox=dict(facecolor="white", alpha=0.95,
boxstyle="round,pad=0.40")

```

```

)

ax_prog.set_xlim(0, 1)
ax_prog.set_ylim(0, 1)
ax_prog.set_xticks([])
ax_prog.set_yticks([])
ax_prog.set_frame_on(True)
ax_prog.set_title("Progress", fontsize=10, pad=2)

progress_bg = Rectangle((0, 0), 1.0, 1.0, color="#f0f0f0")
progress_bar = Rectangle((0, 0), 0.0, 1.0, color="#3A86FF")
ax_prog.add_patch(progress_bg)
ax_prog.add_patch(progress_bar)

def draw_frame(i: int):
    st = frames[i]

    carrot_scatter.set_offsets(st["carrots"] if st["carrots"].size
else np.empty((0, 2)))
    rabbit_scatter.set_offsets(st["rabbits"] if st["rabbits"].size
else np.empty((0, 2)))
    eagle_scatter.set_offsets(st["eagles"] if st["eagles"].size
else np.empty((0, 2)))

    progress = (i + 1) / max(1, len(frames))
    progress_bar.set_width(progress)

    hud_text.set_text(
        "Step: {}/{}\n"
        "Parameters\n"
        "  L={} | p_obs={:.4f}\n"
        "  N_e0={} | N_r0={} | N_c0={}\n"
        "  rho_e={:.3f} | rho_r={:.3f}\n"
        "Populations\n"
        "  Carrots: {}\n"
        "  Rabbits: {}\n"
        "  Eagles: {}".format(
            i + 1, len(frames),
            params_star.L, params_star.p_obs,
            params_star.N_e0, params_star.N_r0, params_star.N_c0,
            params_star.rho_e, params_star.rho_r,
            st["n_carrot"], st["n_rabbit"], st["n_eagle"]
        )
    )

    artists = [carrot_scatter, rabbit_scatter, eagle_scatter,
hud_text, progress_bar]
    if obs_scatter is not None:
        artists.append(obs_scatter)
    return tuple(artists)

```

```

    anim = FuncAnimation(fig, draw_frame, frames=len(frames),
interval=INTERVAL_MS, blit=False, repeat=False)
    writer = FFMpegWriter(
        fps=fps,
        codec="libx264",
        bitrate=2400,
        extra_args=["-pix_fmt", "yuv420p", "-movflags", "+faststart"]
    )
    anim.save(out_name, writer=writer, dpi=95,
savefig_kwargs={"facecolor": "white"})
    plt.close(fig)
    print(f"Saved: {out_name}")
    display(Video(out_name, embed=True))

#
=====
=====
# RUN: Baseline -> GA -> Validation -> Render 100-step video
#
=====
=====

theta0 = np.array([32.5, 0.075, float(MIN_EAGLES), float(MIN_RABBITS),
float(MIN_CARROTS), 0.50, 0.50], dtype=float)
base_params, base_final = run_baseline(theta0, seed=1001,
steps=SIM_STEPS)
print(f"Baseline decoded: {base_params}")
print(f"Baseline final: {base_final}")

# TODO: after implementing GA TODOs above, uncomment:
# best_theta, best_stat = run_ga(
#     generations=40,
#     pop_size=60,
#     elite_frac=0.10,
#     tournament_k=3,
#     crossover_rate=0.85,
#     mutation_rate=0.25,
#     S=S_SEEDS,
#     steps=SIM_STEPS,
#     seed=7,
#     reevaluate_elites=True,
#     verbose_every=1
# )
#
# print(f"\nBest decoded: {decode(best_theta)}")
# print(f"Best stats: {best_stat}")
#
# report = validate_solution(best_theta, seeds=20, steps=SIM_STEPS)

```

```
# print(f"\nValidation report: {report}")  
#  
# params_star, W, obs_xy, frames = simulate_frames(best_theta,  
seed=4242, steps=SIM_STEPS)  
# render_video(params_star, W, obs_xy, frames,  
out_name="ecosystem_ga_demo.mp4")
```

Ex3. Establish the comparison between the results obtained using the ES and the GA